2026 SISBID Graphical Models Demo

Genevera I. Allen and Yufeng Liu

Overview

This demo introduces several ways to estimate networks from multivariate data:

1. correlation and thresholded correlation networks,
2. partial-correlation networks (with thresholding),
3. sparse inverse-covariance estimation using the graphical lasso,
4. neighborhood-selection style estimates of GGMs,
5. nonparanormal graphical models using rank-based correlations, and
6. binary graphical models using nodewise logistic regression.

We will use the Sachs et al. flow-cytometry data: $p = 11$ proteins measured in $n = 6466$ single cells.

Load packages and data

```
required_pkgs <- c("igraph", "huge", "glasso", "glmnet", "ggplot2")
missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]

if (length(missing_pkgs) > 0) {
  stop(
    "Please install the following packages before running this demo: ",
    paste(missing_pkgs, collapse = ", ")
  )
}

library(igraph)
library(huge)
library(glasso)
library(glmnet)
library(ggplot2)
```

```
load("UnsupL_SISBID_2026.Rdata")

p <- ncol(sachsdat)
n <- nrow(sachsdat)

cat("Number of cells:", n, "\n")
```

```
## Number of cells: 7466
```

```
cat("Number of proteins:", p, "\n")
```

```
## Number of proteins: 11
```

```
dim(sachsdat)
```

```
## [1] 7466   11
```

```r
head(sachsdat)
```

```
##            praf       pmek      plcg      PIP2       PIP3        P44     pakts
## [1,]  -97.67193 -132.18100 -46.03364 -132.8207  31.765040 -20.021190 -64.16721
## [2,]  -88.17193 -128.88100 -42.55364 -134.3207 -18.904960  -8.031193 -48.66721
## [3,]  -64.67193 -101.28100 -40.25364 -140.9207 -14.034960 -11.731190 -48.66721
## [4,]  -51.07193  -62.58096 -31.75364 -137.6207 -25.744960 -20.801190 -69.36721
## [5,]  -90.37193 -125.58100 -49.66364 -141.3907  -2.234962  -5.531193 -35.06721
## [6,] -105.27190 -141.63100 -37.25364 -129.0207 -16.134960 -14.731190 -55.46721
##             PKA       PKC       P38       pjnk
## [1,] -211.75860 -13.34166  -90.1145 -33.267500
## [2,] -273.75860 -26.97166 -118.5145 -11.767500
## [3,] -222.75860 -18.94166 -103.1145 -53.767500
## [4,]  -97.75859 -16.64166 -106.4145 -50.167500
## [5,] -320.75860 -25.68166 -109.3145   8.032497
## [6,]  -15.75859 -16.64166  -85.9145 -15.467500
```

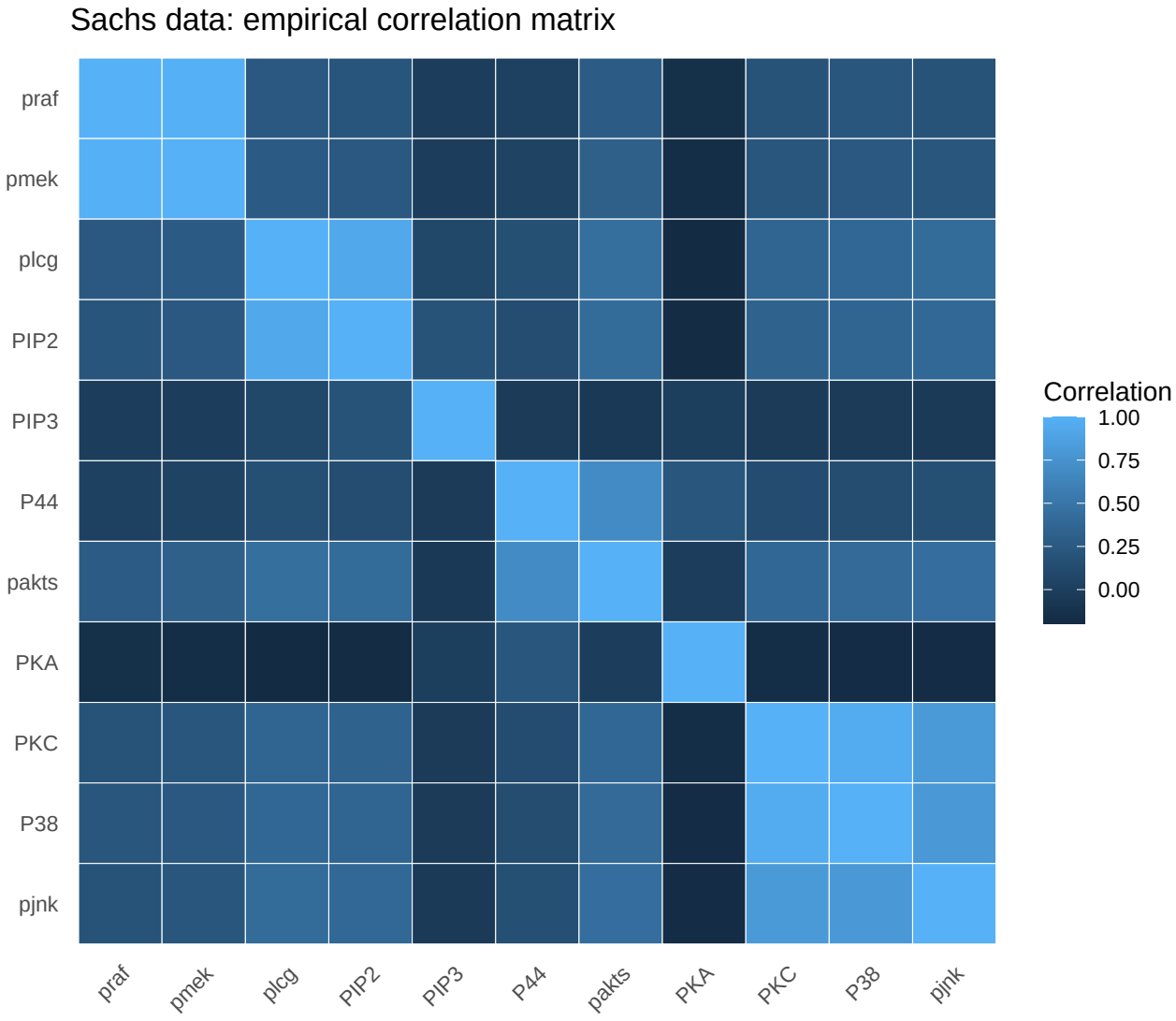
```r
edge_count <- function(adj) {
  sum(adj != 0) / 2
}

make_undirected_graph <- function(adj, node_names = colnames(sachsdat)) {
  adj <- as.matrix(adj)
  diag(adj) <- 0
  adj <- (adj != 0) | t(adj != 0)

  if (is.null(node_names)) {
    node_names <- paste0("V", seq_len(ncol(adj)))
  }

  graph <- graph_from_adjacency_matrix(
    adj,
    mode = "undirected",
    diag = FALSE
  )
  V(graph)$name <- node_names
  graph
}

plot_network <- function(graph, title, layout_matrix = NULL) {
  if (is.null(layout_matrix)) {
    layout_matrix <- layout_in_circle(graph)
  }

  plot(
    graph,
    layout = layout_matrix,
    main = paste0(title, "\n", ecount(graph), " edges"),
    vertex.size = 26,
    vertex.color = "skyblue",
    vertex.frame.color = "gray40",
    vertex.label.color = "black",
    vertex.label.cex = 0.9,
    edge.color = "gray40",
```

```
    edge.width = 2
  )
}

plot_matrix_heatmap <- function(mat, title, fill_label = "Value") {
  mat <- as.matrix(mat)
  mat_df <- as.data.frame(as.table(mat))
  names(mat_df) <- c("Row", "Column", "Value")

  ggplot(mat_df, aes(x = Column, y = Row, fill = Value)) +
    geom_tile(color = "white") +
    scale_y_discrete(limits = rev(levels(mat_df$Row))) +
    labs(title = title, x = NULL, y = NULL, fill = fill_label) +
    theme_minimal() +
    theme(
      axis.text.x = element_text(angle = 45, hjust = 1),
      panel.grid = element_blank()
    )
}

partial_cor_from_cor <- function(cor_mat) {
  precision <- solve(cor_mat)
  partial_cor <- -cov2cor(precision)
  diag(partial_cor) <- 1
  partial_cor
}

adjacency_from_precision <- function(precision, tol = 1e-6) {
  adj <- abs(precision) > tol
  diag(adj) <- FALSE
  adj
}
```

Correlation network

A simple first network is a **coexpression network**: connect two proteins when their empirical correlation is large in absolute value. This uses marginal correlations, so an edge can reflect either a direct relationship or an indirect relationship mediated through other proteins.

```
sachscor <- cor(sachsdat)

plot_matrix_heatmap(
  sachscor,
  title = "Sachs data: empirical correlation matrix",
  fill_label = "Correlation"
)
```



Simple thresholding of correlations

Here we choose a correlation threshold by hand. In practice, the threshold controls the density of the graph and should be varied as a sensitivity analysis.

```
tau <- 0.10

A_cor_threshold <- abs(sachscor) > tau
diag(A_cor_threshold) <- FALSE

g_cor_threshold <- make_undirected_graph(A_cor_threshold)
edge_count(A_cor_threshold)
```

```
## [1] 43
```

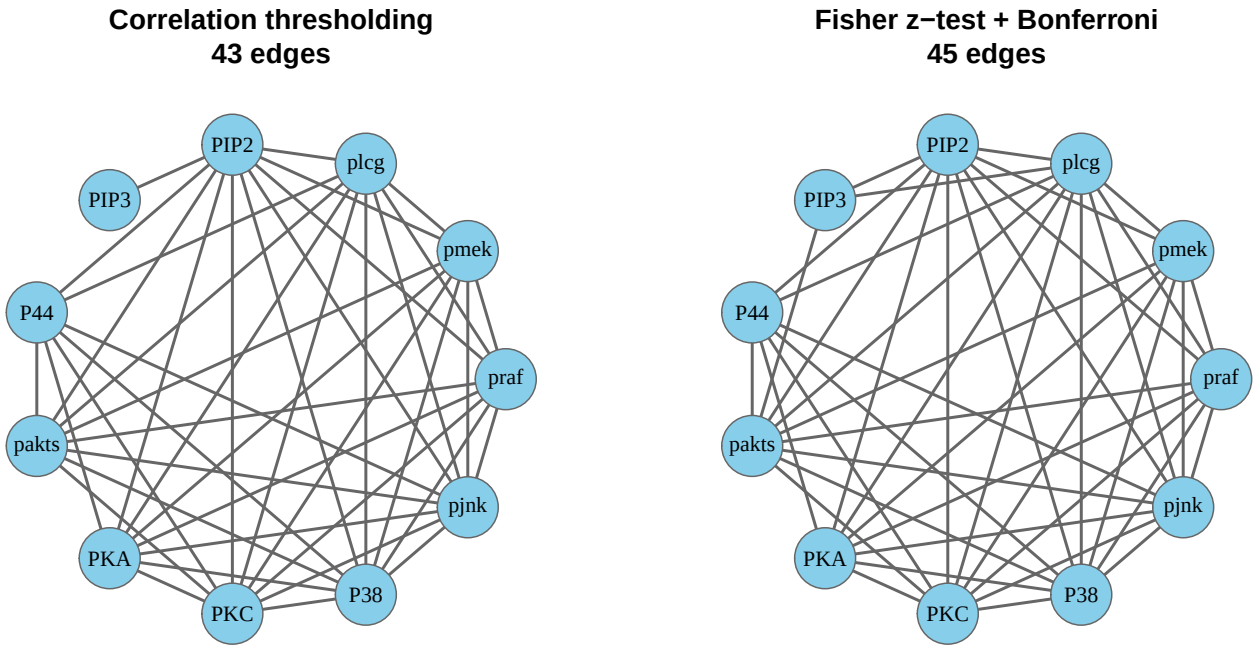
Testing for nonzero correlations

We can also use Fisher's z-transform to test whether each correlation is nonzero, then apply a Bonferroni correction over the $p(p-1)/2$ protein pairs.

```r
fisher_z <- atanh(sachscor)
fisher_p <- 2 * pnorm(abs(fisher_z), mean = 0, sd = 1 / sqrt(n - 3), lower.tail = FALSE)

diag(fisher_p) <- NA

alpha <- 0.01
m_tests <- p * (p - 1) / 2

A_cor_test <- fisher_p < alpha / m_tests
A_cor_test[is.na(A_cor_test)] <- FALSE
diag(A_cor_test) <- FALSE

g_cor_test <- make_undirected_graph(A_cor_test)
edge_count(A_cor_test)
```

```
## [1] 45
```

```r
layout_sachs <- layout_in_circle(g_cor_threshold)

par(mfrow = c(1, 2), mar = c(1, 1, 4, 1))
plot_network(g_cor_threshold, "Correlation thresholding", layout_sachs)
plot_network(g_cor_test, "Fisher z-test + Bonferroni", layout_sachs)
```




```r
par(mfrow = c(1, 1))
```

Partial-correlation network

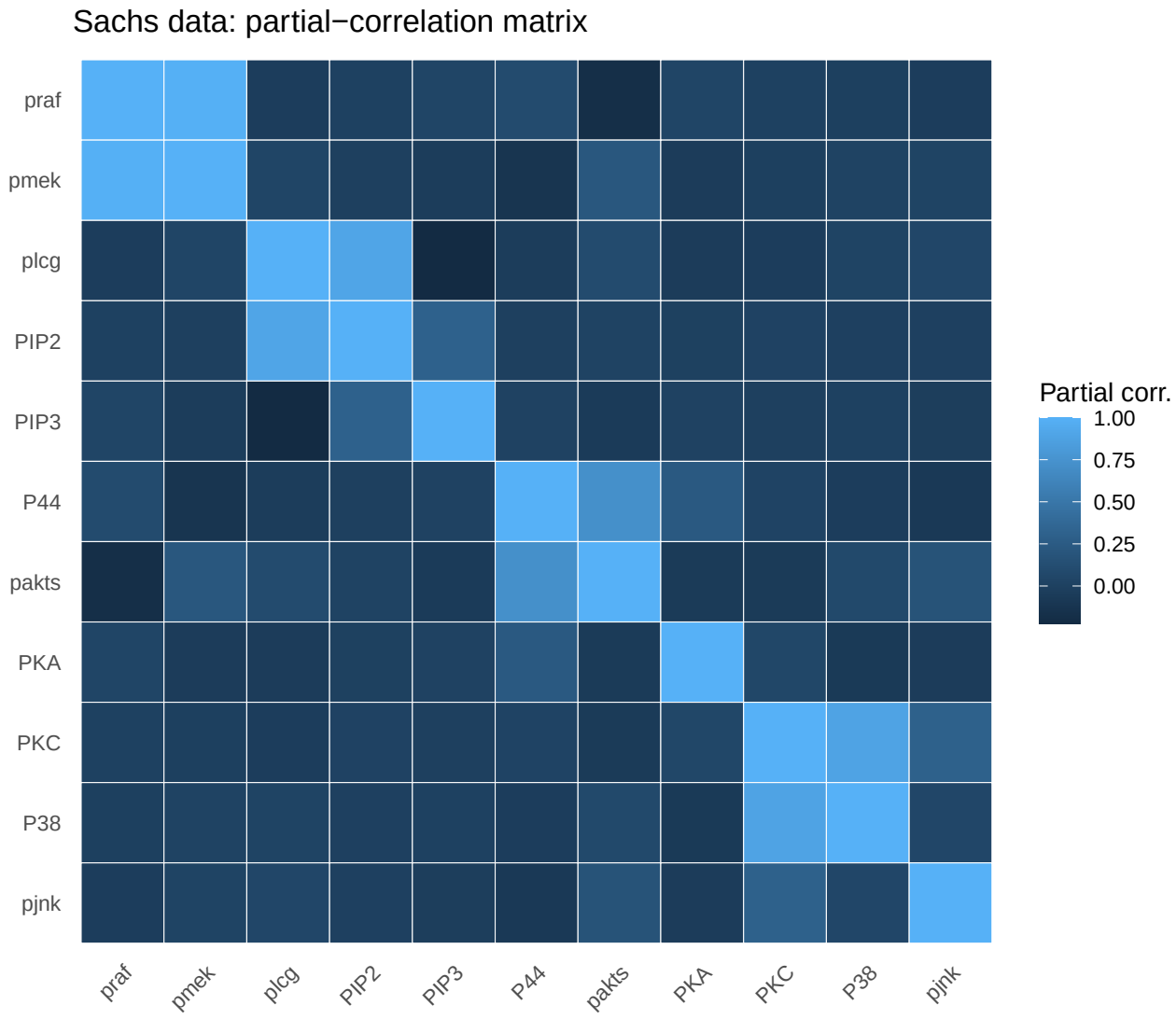
A partial correlation measures the association between two variables after linearly adjusting for all other variables. Under a Gaussian graphical model, zero partial correlation corresponds to conditional independence.

```r
partial_cor <- partial_cor_from_cor(sachscor)
partial_threshold <- 0.05

A_partial <- abs(partial_cor) > partial_threshold
```

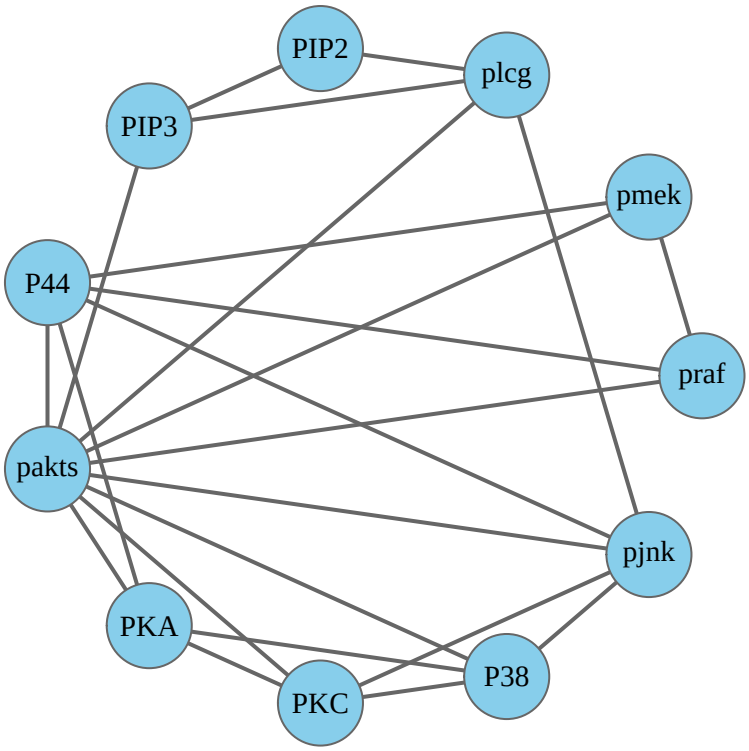
```r
diag(A_partial) <- FALSE

g_partial <- make_undirected_graph(A_partial)
edge_count(A_partial)
```

```
## [1] 23
```

```r
plot_matrix_heatmap(
  partial_cor,
  title = "Sachs data: partial-correlation matrix",
  fill_label = "Partial corr."
)
```




```r
plot_network(g_partial, "Thresholded Partial-correlation network", layout_sachs)
```



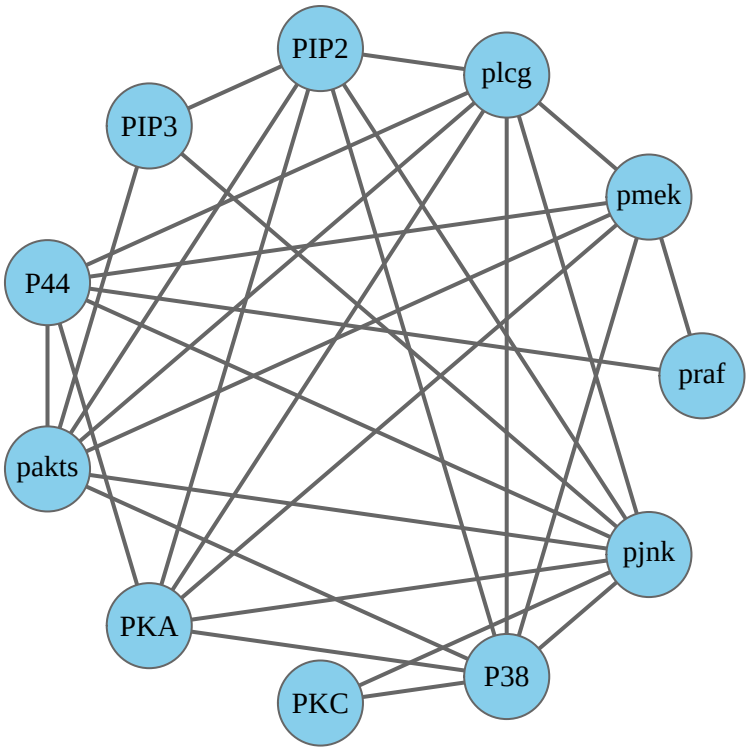
Graphical Lasso

The graphical lasso estimates a sparse inverse covariance matrix by penalizing the off-diagonal entries of the precision matrix. Larger values of lambda produce sparser networks.

```
lambda = .05

glasso_est <- glasso(
  s = sachscor,
  rho = lambda,
  approx = FALSE,
  penalize.diagonal = FALSE
)

A_glasso <- adjacency_from_precision(glasso_est$wi)
g_glasso <- make_undirected_graph(A_glasso)
edge_count(A_glasso)
```

```
## [1] 30
```

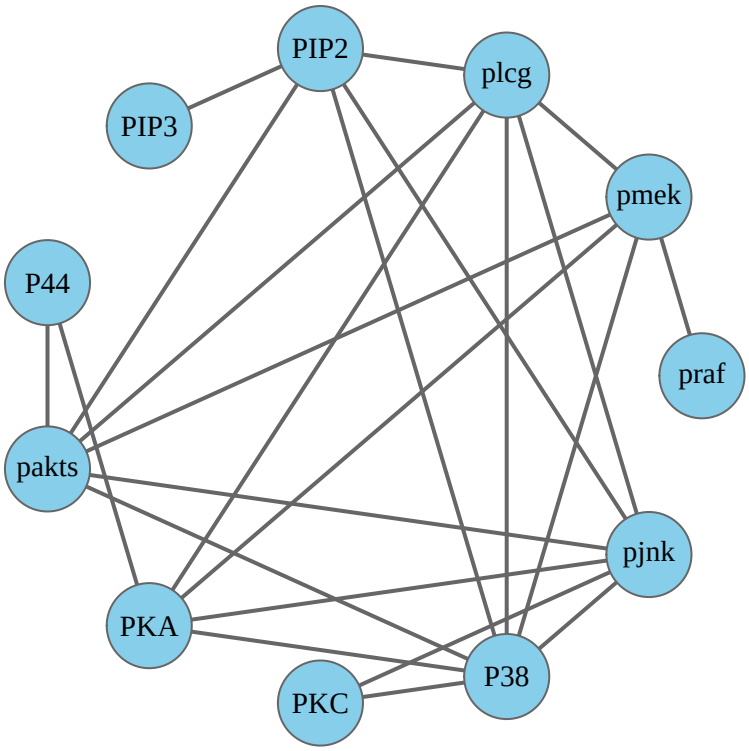
```
plot_network(g_glasso, "Graphical lasso", layout_sachs)
```




```
lambda = .1

glasso_est <- glasso(
  s = sachscor,
  rho = lambda,
  approx = FALSE,
  penalize.diagonal = FALSE
)

A_glasso <- adjacency_from_precision(glasso_est$wi)
g_glasso <- make_undirected_graph(A_glasso)
edge_count(A_glasso)
```

```
## [1] 23
```

```
plot_network(g_glasso, "Graphical lasso: Larger Lambda", layout_sachs)
```



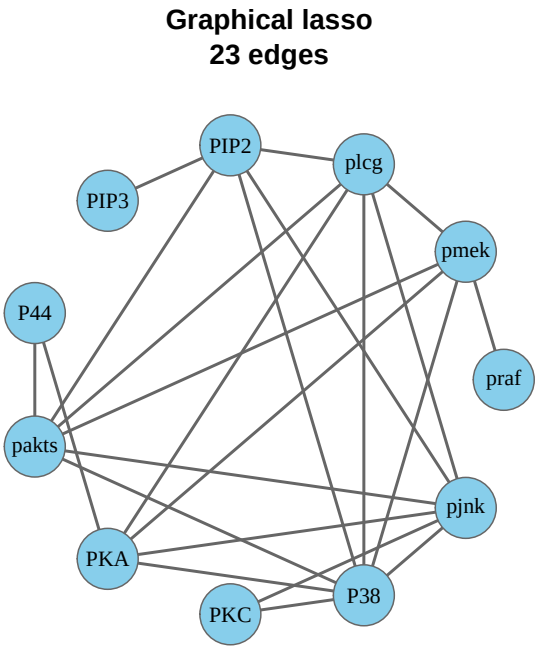
Neighborhood-selection style GGM estimate

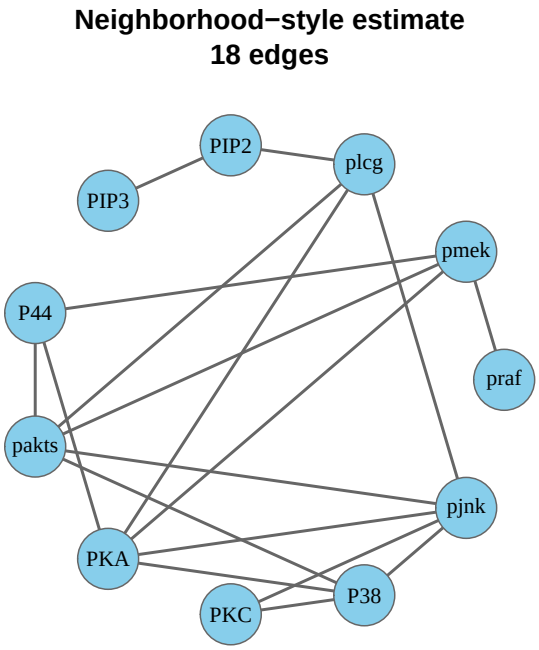
Neighborhood selection estimates the neighbors of each node using sparse regression. The `huge` package implements model-selection tools for this approach. The original demo also used `glasso(..., approx = TRUE)` as a fast neighborhood-selection style approximation, shown here for comparison.

```
ns_est <- glasso(
  s = sachscor,
  rho = lambda,
  approx = TRUE,
  penalize.diagonal = FALSE
)

A_ns <- adjacency_from_precision(ns_est$wi)
g_ns <- make_undirected_graph(A_ns)
edge_count(A_ns)
```

```
## [1] 13.5
```

```
par(mfrow = c(1, 2), mar = c(1, 1, 4, 1))
plot_network(g_glasso, "Graphical lasso", layout_sachs)
plot_network(g_ns, "Neighborhood-style estimate", layout_sachs)
```



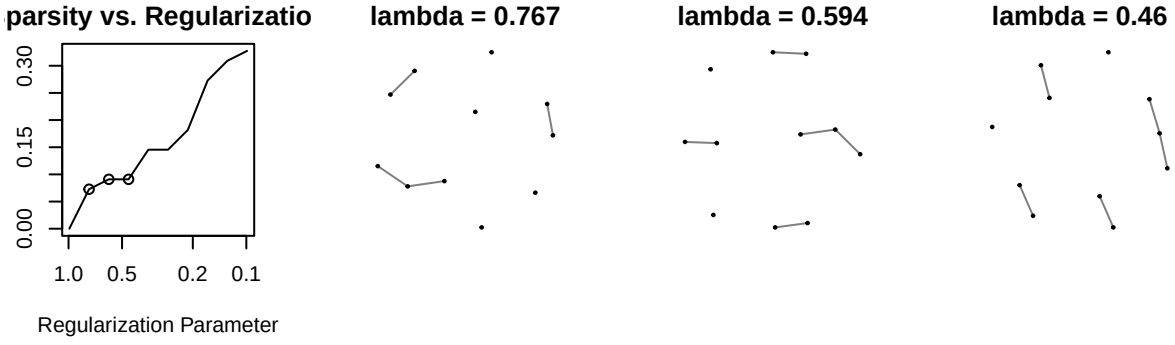



```
par(mfrow = c(1, 1))
```

Optional: Neighborhood selection with `huge`

The `huge` package fits a full path of neighborhood-selection estimates over many values of the regularization parameter. Stability selection can then choose a point along that path.

```
X <- data.matrix(scale(sachsdat))

net_path <- huge(
  X,
  method = "mb",
  verbose = FALSE
)

plot(net_path)
```

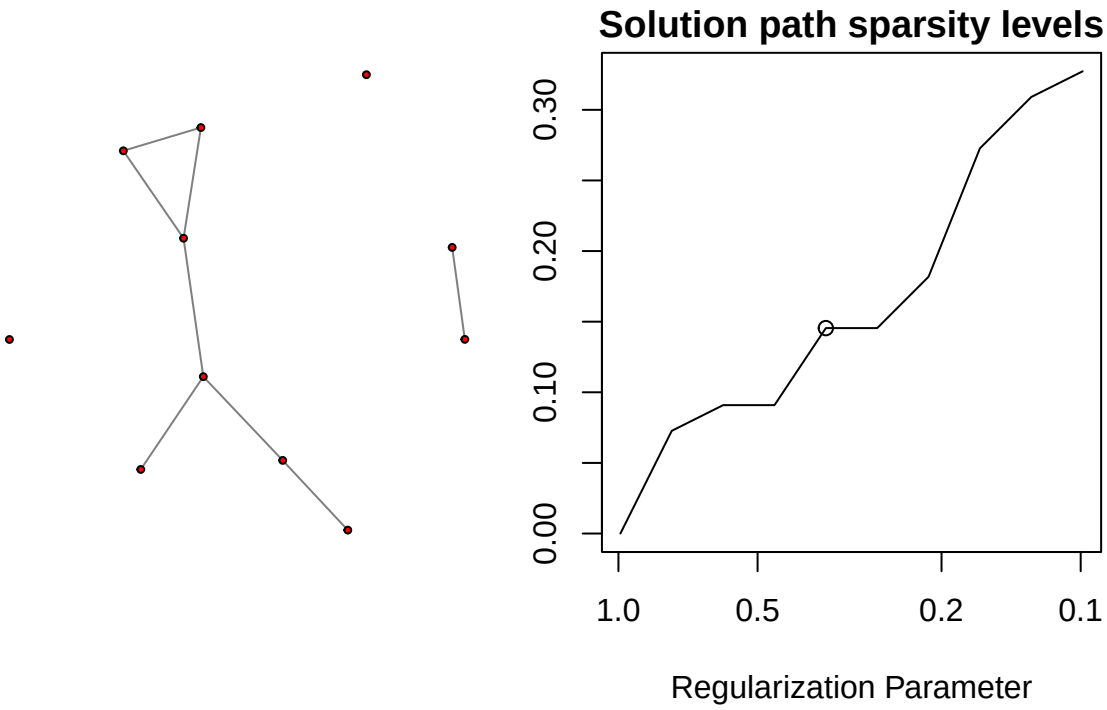



```
net_stars <- huge.select(
  net_path,
  criterion = "stars",
  verbose = FALSE
)
```

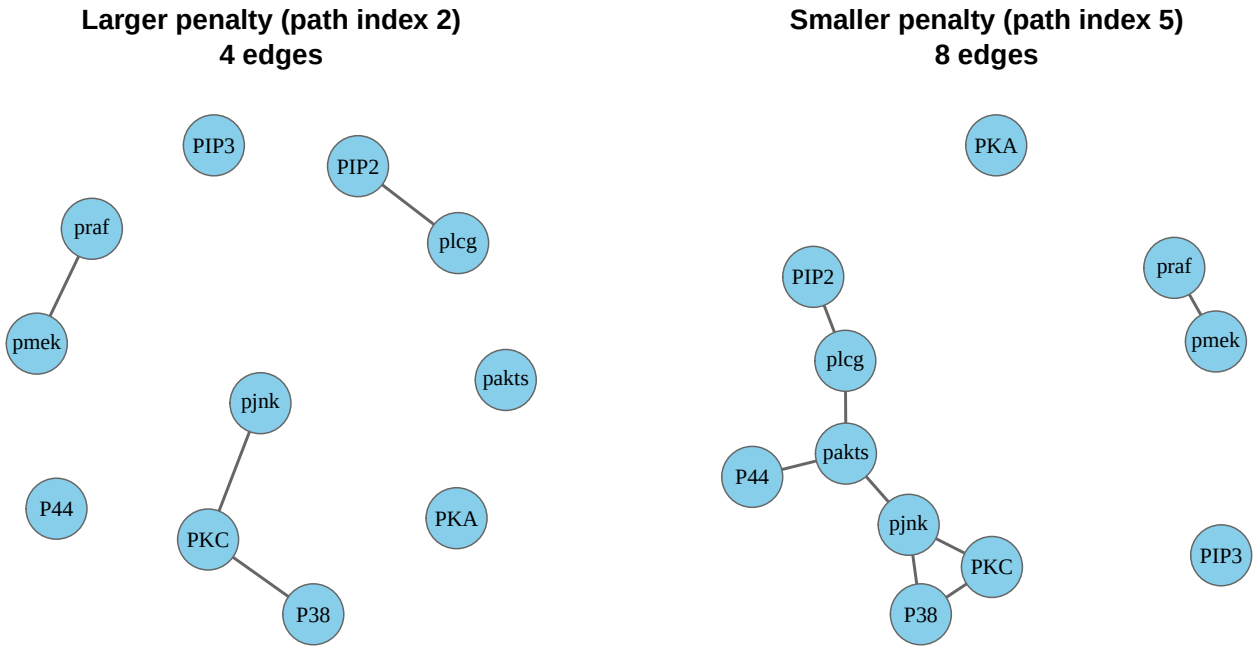
```
net_stars
```

```
## Model: Meinshausen & Buhlmann Graph Estimation (mb)
## selection criterion: stars
## Graph dimension: 11
## sparsity level 0.1454545
```

```
plot(net_stars)
```




```
plot_huge_graph_at_path <- function(net_path, path_index, title) {
  path_index <- min(path_index, length(net_path$path))
  mat <- net_path$path[[path_index]]
  graph <- graph_from_adjacency_matrix(
    mat,
    mode = "undirected",
    diag = FALSE
  )
  V(graph)$name <- colnames(X)

  plot_network(
    graph,
    paste0(title, " (path index ", path_index, ")"),
    layout_with_kk(graph)
  )
}

par(mfrow = c(1, 2), mar = c(1, 1, 4, 1))
plot_huge_graph_at_path(net_path, 2, "Larger penalty")
plot_huge_graph_at_path(net_path, 5, "Smaller penalty")
```




```
par(mfrow = c(1, 1))
```

Nonparanormal graphical models

Gaussian graphical models can be extended using rank-based transformations. The nonparanormal model estimates dependence after allowing monotone transformations of the observed variables.

Rank-based correlation using Spearman correlations

```
spearman_cor <- cor(sachsdat, method = "spearman")
npn_cor_spearman <- 2 * sin(spearman_cor * pi / 6)

npn_est_spearman <- glasso(
  s = npn_cor_spearman,
  rho = lambda,
  approx = FALSE,
  penalize.diagonal = FALSE
)

A_npn_spearman <- adjacency_from_precision(npn_est_spearman$wi)
g_npn_spearman <- make_undirected_graph(A_npn_spearman)
edge_count(A_npn_spearman)
```

```
## [1] 32
```

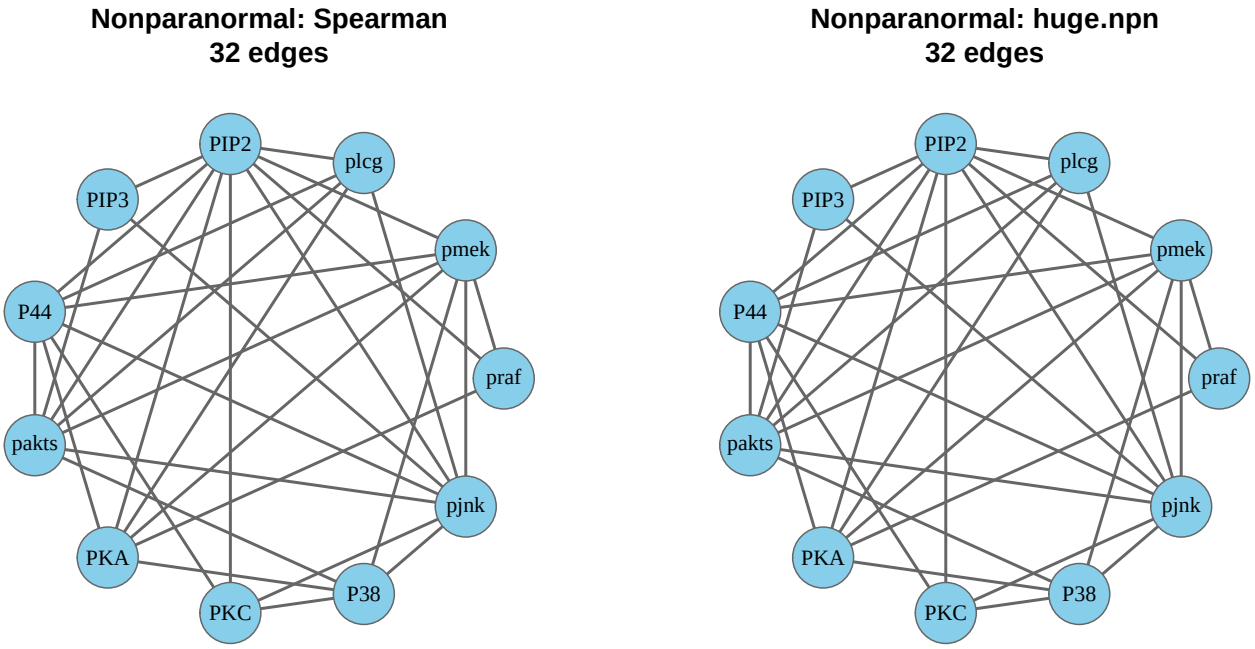
Nonparanormal correlation using `huge.npn`

```
npn_cor_huge <- huge.npn(
  x = sachsdat,
  npn.func = "skeptic",
  npn.thresh = NULL,
  verbose = FALSE
)
```

```
npn_est_huge <- glasso(
  s = npn_cor_huge,
  rho = lambda,
  approx = FALSE,
  penalize.diagonal = FALSE
)

A_npn_huge <- adjacency_from_precision(npn_est_huge$wi)
g_npn_huge <- make_undirected_graph(A_npn_huge)
edge_count(A_npn_huge)
```

```
## [1] 32
```

```
par(mfrow = c(1, 2), mar = c(1, 1, 4, 1))
plot_network(g_npn_spearman, "Nonparanormal: Spearman", layout_sachs)
plot_network(g_npn_huge, "Nonparanormal: huge.npn", layout_sachs)
```




```
par(mfrow = c(1, 1))
```

Binary network estimation

For binary data, one simple approach is nodewise logistic regression: for each protein, regress its binary state on the other proteins and record which predictors are selected. We symmetrize the resulting directed neighborhoods using the OR rule: an undirected edge is present if either node selects the other.

```
sachsbin <- ifelse(sachsdat > 0, 1, 0)
head(sachsbin)
```

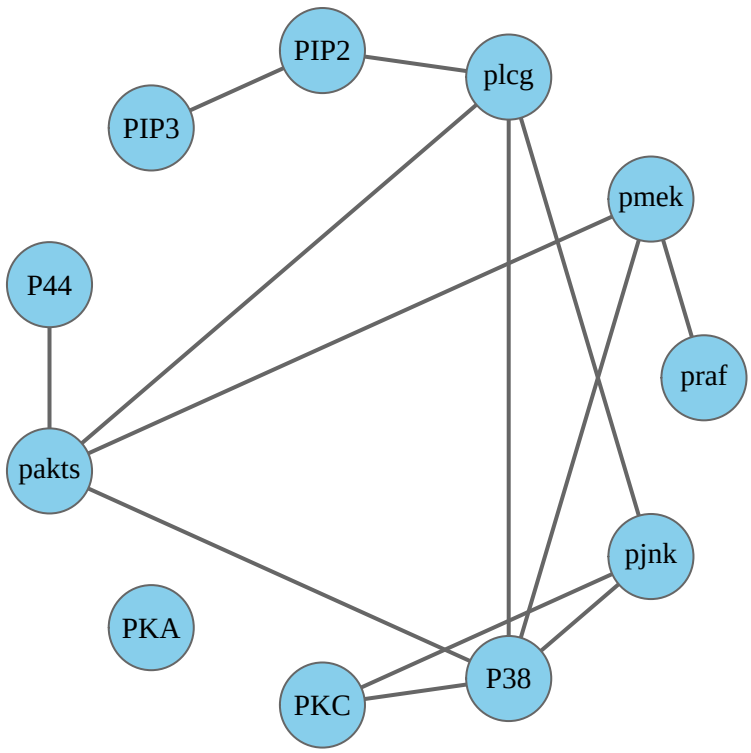
```
##      praf pmek plcg PIP2 PIP3 P44 pakts PKA PKC P38 pjnk
## [1,]    0    0    0    0    1   0     0   0   0   0    0
## [2,]    0    0    0    0    0   0     0   0   0   0    0
## [3,]    0    0    0    0    0   0     0   0   0   0    0
## [4,]    0    0    0    0    0   0     0   0   0   0    0
## [5,]    0    0    0    0    0   0     0   0   0   0    1
```

```
## [6,]    0    0    0    0    0    0    0    0    0    0    0
```

```
bin_est <- matrix(0, nrow = p, ncol = p)
colnames(bin_est) <- rownames(bin_est) <- colnames(sachsdat)

for (j in seq_len(p)) {
  y_j <- sachsbin[, j]

  if (length(unique(y_j)) < 2) {
    next
  }

  nbr <- glmnet(
    x = as.matrix(sachsbin[, -j, drop = FALSE]),
    y = y_j,
    family = "binomial",
    lambda = lambda,
    standardize = TRUE
  )

  beta_j <- as.matrix(coef(nbr))[-1, 1]
  bin_est[j, -j] <- as.numeric(abs(beta_j) > 0)
}

A_binary <- (bin_est + t(bin_est)) > 0
diag(A_binary) <- FALSE

g_binary <- make_undirected_graph(A_binary)
edge_count(A_binary)
```

```
## [1] 13
```

```
plot_network(g_binary, "Binary nodewise logistic network", layout_sachs)
```



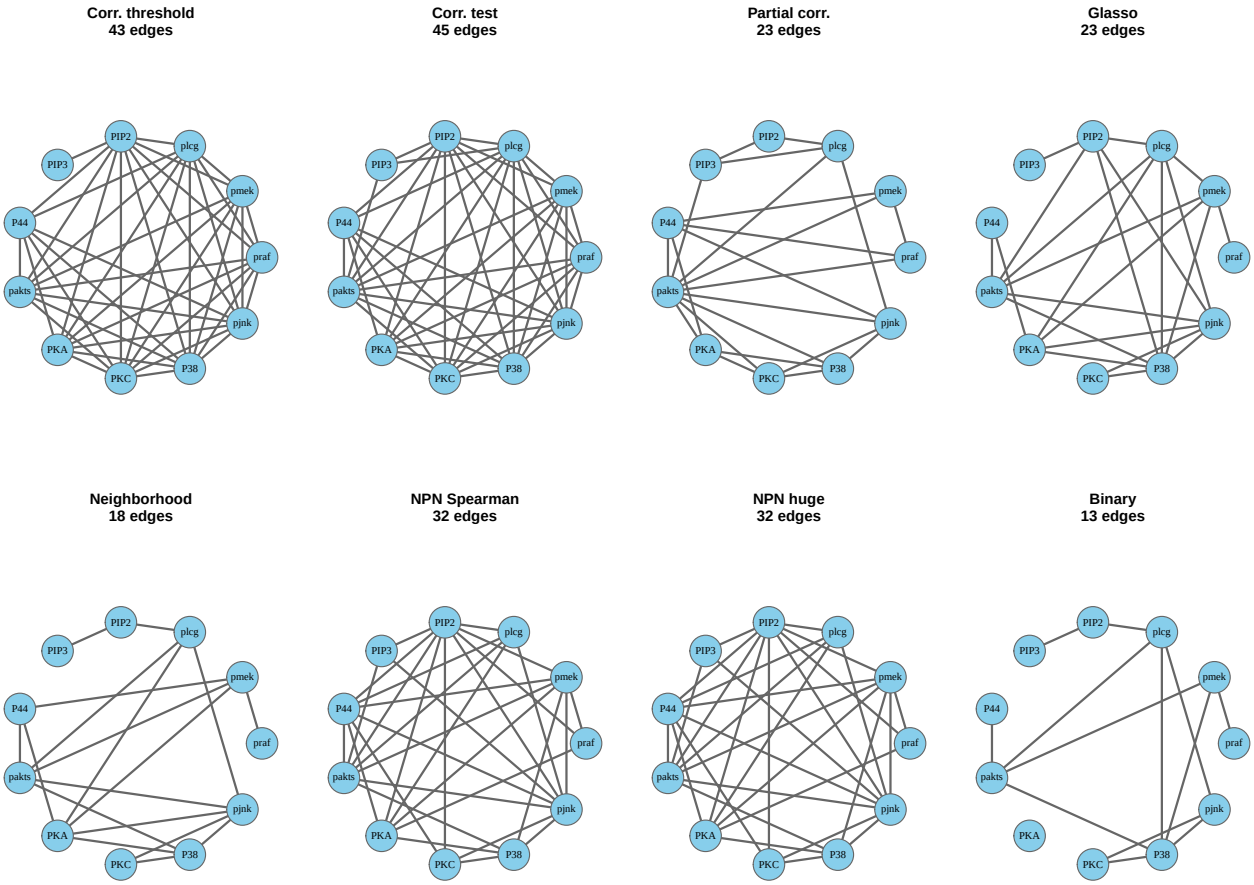
Compare all network estimates

```
network_summary <- data.frame(
  Method = c(
    "Correlation threshold",
    "Fisher z-test + Bonferroni",
    "Partial correlation",
    "Graphical lasso",
    "Neighborhood-style estimate",
    "Nonparanormal: Spearman",
    "Nonparanormal: huge.npn",
    "Binary logistic"
  ),
  Edges = c(
    ecount(g_cor_threshold),
    ecount(g_cor_test),
    ecount(g_partial),
    ecount(g_glasso),
    ecount(g_ns),
    ecount(g_npn_spearman),
    ecount(g_npn_huge),
    ecount(g_binary)
  )
)
```

```
knitr::kable(network_summary)
```

Method	Edges
Correlation threshold	43
Fisher z-test + Bonferroni	45
Partial correlation	23
Graphical lasso	23
Neighborhood-style estimate	18
Nonparanormal: Spearman	32
Nonparanormal: huge.npn	32
Binary logistic	13

```
par(mfrow = c(2, 4), mar = c(1, 1, 4, 1))
plot_network(g_cor_threshold, "Corr. threshold", layout_sachs)
plot_network(g_cor_test, "Corr. test", layout_sachs)
plot_network(g_partial, "Partial corr.", layout_sachs)
plot_network(g_glasso, "Glasso", layout_sachs)
plot_network(g_ns, "Neighborhood", layout_sachs)
plot_network(g_npn_spearman, "NPN Spearman", layout_sachs)
plot_network(g_npn_huge, "NPN huge", layout_sachs)
plot_network(g_binary, "Binary", layout_sachs)
```



```r
par(mfrow = c(1, 1))
```